Final Project Report

The Hive Platform
Community Service Exchange TimeBank Platform

Ayşenur Ünal **Sena Öz**
Kenan Altunbaş **Yusuf Savaş**

Software Engineering MS Program
Boğaziçi University

SWE 574
Software Development Practice
Spring 2026

May 2026

Git Repository:	https://github.com/senaoz/SWE-574
Git Version:	v0.9
Deployment URL:	https://swe.gnahh5.easypanel.host
Backend API:	https://backend-swe.gnahh5.easypanel.host/docs

Contents

List of Figures

List of Tables

Chapter 1

List of Deliverables

This chapter lists all project deliverables submitted across the two milestones and the final report.

Deliverable	**Location**
Software Requirements Specification	https://github.com/senaoz/SWE-574/wiki/SRS
UML Diagrams	https://github.com/senaoz/SWE-574/wiki/UML-Diagrams
Scenarios & Mockups	https://github.com/senaoz/SWE-574/wiki/Scenarios-&-Mockups
Project Plan	https://github.com/senaoz/SWE-574/wiki
Communication Plan	https://github.com/senaoz/SWE-574/wiki/Communication-Plan
Responsibility Assignment Matrix	https://github.com/senaoz/SWE-574/wiki/Responsibility-Assignment-Matrix-(RACI)
Weekly Reports & Meeting Notes	https://github.com/senaoz/SWE-574/wiki/Meeting-Notes
Final Project Report (this document)	https://github.com/senaoz/SWE-574/tree/main/final-deliverables
Source Code Repository	https://github.com/senaoz/SWE-574
Live Web Deployment	https://swe.gnahh5.easypanel.host
Android APK	https://github.com/senaoz/SWE-574/tree/main/apk
Demo Video	*https://drive.google.com/file/d/1BxsD2vjVa1TiVinZe4NU7uNgSqXYdloy/view?usp=sharing*
Backend API Documentation	https://backend-swe.gnahh5.easypanel.host/docs
Updated SRS	https://github.com/senaoz/SWE-574/wiki/SRS
Architecture & Design Docs	https://github.com/senaoz/SWE-574/wiki/UML-Diagrams
AI Use Declaration	https://github.com/senaoz/SWE-574/blob/main/AI_USAGE.md

Chapter 2

Test Credentials and Demo

2.1 Demo Video

https://drive.google.com/file/d/1BxsD2vjVa1TiVinZe4NU7uNgSqXYdloy/view?usp=sharing

2.2 Test User Accounts

The following test accounts are available on the live deployment for evaluation:

Role	Email	Password
Admin	<tazeyta@gmail.com>	Password123
Regular User	<elif.sahin@example.com>	Password123
Regular User	<mehmet.demir@example.com>	Password123

Table 2.1: Test User Credentials

2.3 Testing Instructions

1. Navigate to https://swe.gnahh5.easypanel.host
2. Log in with a test account from the table above
3. Key features to test:
    - Service creation (offers and needs)
    - Service discovery, search, and map view
    - Join request workflow (apply, approve, reject)
    - Transaction completion with dual confirmation
    - Chat functionality
    - TimeBank balance and history
    - Forum (discussions, events, communities)
    - Badges and ratings
    - Admin panel (using admin account)

Chapter 3

Project Overview

3.1 Introduction

The Hive Platform is a community-oriented service exchange platform based on the TimeBank concept, where one hour of service equals one hour of credit. The platform enables community members to offer services they can provide, request services they need, and exchange services using a TimeBank credit system.

The project was developed as part of the SWE 574 Software Development Practice course at Boğaziçi University, Spring 2026 semester.

3.2 Key Features

- **Service Exchange:** Users create service offers and needs; join requests, approvals, and dual-confirmation transactions manage the exchange workflow.
- **TimeBank System:** Credits in hours track contributions and consumption. Starting balance: 3 hours; maximum: 10 hours.
- **Interactive Map:** Google Maps API visualizes service locations with privacy-preserving fuzzed coordinates.
- **Forum:** Discussions, events, and community spaces for community interaction.
- **Communities:** User-created groups with posts, events, and member management.
- **Chat:** Messaging between service participants (auto-created on approval).
- **Badges:** Automatically awarded based on objective milestones.
- **Ratings:** Post-exchange star ratings with feedback labels and image attachments.
- **Recommendations:** Tag-based personalized service recommendations.
- **Admin Panel:** Moderation, user management, analytics, and report handling.
- **Android App:** Native Android client (Kotlin/Jetpack Compose) sharing the same backend.

3.3 Technology Stack

3.3.1 Frontend (Web)

React 18 with TypeScript, built with Vite. UI components from Radix UI, styled with Tailwind CSS. Google Maps API for map interaction. React Query for server state; Axios for API calls.

3.3.2 Backend

FastAPI (Python 3.11) with async Motor driver for MongoDB. JWT authentication (HS256, 30-minute expiry). Pydantic models for request/response validation. Content moderation via `alt-profanity-check`. WikiData API for semantic tag suggestions.

3.3.3 Database

MongoDB 7 with geospatial (2dsphere) indexes on service locations. Motor async driver. 15 collections including users, services, transactions, forum, communities, and notifications.

3.3.4 Android App

Kotlin with Jetpack Compose, MVVM architecture, Hilt DI, Retrofit + Moshi, DataStore for token persistence.

3.3.5 Infrastructure

Docker + Docker Compose for full-stack containerization. Nginx serves the React SPA in production. Deployed on EasyPanel via GitHub Actions CI/CD.

3.4 Architecture

The system follows a three-tier architecture:

- **Client tier:** React SPA (web) and Android app
- **API tier:** FastAPI REST API with JWT authentication, served behind Nginx
- **Data tier:** MongoDB 7 database

See Chapter 5 for detailed architecture diagrams.

Chapter 4

Software Requirements Specification

Note: This SRS was authored during the design phase and is based on IEEE 830. The final implementation uses FastAPI + MongoDB rather than the originally planned Django/Spring Boot + PostgreSQL stack. All functional requirements were implemented as specified.

4.1 Introduction

4.1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define, in detail, the functional and non-functional requirements of The Hive platform, a community-oriented, open-source web application designed for the mutual exchange of services based on time rather than money.

This document serves as a reference for:

- Developers, who will design, implement, and maintain the platform.
- Project stakeholders, who will ensure that the product aligns with its vision of inclusivity and cooperation.
- Testers and quality engineers, who will verify that the delivered software meets the stated requirements.
- Community moderators and administrators, who will oversee system behavior and enforce community standards.

4.1.2 Product Scope

The Hive is a virtual public space that enables individuals to exchange services in a non-commercial, community-driven environment. Its main objectives are:

- To provide a map-based interface for posting and discovering service Offers and Needs.
- To enable semantic tagging for easy discovery and organization of activities.
- To maintain a TimeBank system where time, rather than money, serves as the unit of value.
- To support profile-based community trust, comments, and transparent histories of contribution.
- To allow users to connect, communicate, and schedule services through a simple, privacy-aware interface.

The primary goals of The Hive are to cultivate mutual support within local and remote communities, ensure fairness through a balanced exchange of time, maintain transparency and simplicity while minimizing barriers to participation, and promote sustainability through open contribution and community moderation.

4.1.3 Definitions, Acronyms, and Abbreviations

Refer to the Glossary in Section 4.6 for all terms, acronyms, and abbreviations used throughout this document.

4.1.4 References

- IEEE 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Wikidata Ontology for Semantic Tagging (https://www.wikidata.org/).
- Docker Documentation (https://docs.docker.com/).

4.1.5 Overview

The remainder of this document is organized as follows:

- Section 4.2 — Product Overview: general factors and contextual information.
- Section 4.3 — Functional Requirements (FR–1 through FR–19).
- Section 4.4 — Non-Functional Requirements.
- Section 4.5 — External Interfaces.
- Section 4.6 — Glossary.

4.2 Product Overview

4.2.1 Product Perspective

The Hive is a new, self-contained, open-source web platform designed to facilitate community-based service exchange through time-based reciprocity. It does not replace or extend an existing commercial system; rather, it introduces a non-monetary, trust-driven alternative to traditional gig or marketplace applications.

The platform is composed of modular subsystems connected through a RESTful backend API:

- **Frontend Web Application:** A responsive, map-based interface that allows users to create, browse, and manage Offers and Needs, interact through messaging, and track their TimeBank balance.
- **Backend Service Layer:** Exposes RESTful endpoints for authentication, user management, offers, needs, messaging, moderation, and TimeBank transactions.
- **Database Layer:** Stores user profiles, posts, tags, transaction histories, and moderation data.
- **Containerization and Deployment:** The application is deployed using Docker, enabling consistent environment setup and portability.

4.2.2 Product Functions

At a high level, The Hive provides the following major functional capabilities:

- **User Registration and Authentication:** Register new users, log in via secure credentials, manage profiles and TimeBank balance.
- **Offers and Needs Management:** Create, edit, cancel, and delete offers and needs; assign semantic tags and optional approximate location; specify capacity.
- **Handshake Mechanism:** Allow users to send offers of help with optional messages; require explicit acceptance to finalize exchanges.
- **Messaging System:** Enable text-based communication between participants of an active exchange.
- **TimeBank Accounting:** Track contribution and consumption in hours; enforce the 10-hour reciprocity limit.
- **Map-Based Discovery:** Visualize offers and needs geographically; filter by tags, distance, or online availability.
- **Profile and Comment System:** Publicly display participation history, tags, and old exchanges; allow moderated comment posting after exchange completion; display user badges.
- **Badges and Recognition:** Award badges automatically when users meet objective milestones; show badges on profiles and next to display names.
- **Admin and Moderation Tools:** Admins manage users, moderators, tags, and reported content; moderators handle content review and report resolution.
- **Community Forum:** Provide a space for community-wide discussions and event announcements with two tabs: Discussions and Events.

4.2.3 User Characteristics

User Class	Description	Privileges
Regular User	General community member who posts, offers, and participates in exchanges. Basic web skills.	Create offers/needs, message, comment, report content.
Moderator	Trusted user responsible for maintaining community quality and handling reports.	Remove content, review flagged posts, manage comments, resolve reports.
Admin	Platform operator overseeing system health, data consistency, and community governance.	Full control: manage users, moderators, tags, reports, and system settings.

Table 4.1: User Classes

4.2.4 Constraints

Design and Implementation Constraints

- The system must be web-based during the initial release.
- The backend must provide RESTful APIs.
- The application must be Dockerized for deployment.
- No external authentication (OAuth).

Functional Constraints

- Each user starts with 3 TimeBank hours.
- When a user reaches a 10-hour surplus, they must create a Need to restore balance.
- Location data must be approximate only; no exact addresses are stored.
- Communication is text-only; no video or audio integration.
- Each exchange must involve a handshake for confirmation.
- Offers and Needs may specify a capacity > 1 (default 1); the system shall prevent accepting more participants than the declared capacity.

Community and Governance Constraints

- Comments are moderated by an automated filter.
- Community guidelines define acceptable behavior.
- Reports of inappropriate content must be reviewable by moderators.

4.2.5 Assumptions and Dependencies

Assumptions

- Users act in good faith and understand the community guidelines.
- Internet connectivity is stable enough for interactive map and messaging operations.
- Users voluntarily provide approximate location or mark their service as remote.
- The majority of services are non-monetary and informal.

Dependencies

- A functional mapping API (Google Maps API) for displaying offers and needs.
- A stable Docker runtime environment for deployment.
- Wikidata API for semantic tag suggestions and validation.

4.3 Functional Requirements

4.3.1 FR–1: User Registration and Authentication

- FR–1.1 The system shall allow users to register with a unique username, email, and password.
- FR–1.2 The system shall validate email format and password complexity upon registration.
- FR–1.3 The system shall provide login and logout functionality.
- FR–1.4 The system shall encrypt all stored passwords using secure hashing.
- FR–1.5 The system shall provide session-based authentication for authorized access.
- FR–1.6 The system shall prompt users to select at least one interest (semantic tag) during the registration process to personalize their profile experience.

4.3.2 FR–2: Profile Management

- FR–2.1 Each user shall have a public profile displaying their interest tags, total hours given/received, badges, completed exchanges, and comments.
- FR–2.2 The system shall display a user's current TimeBank balance on their profile.
- FR–2.3 The profile shall show completed exchanges.
- FR–2.4 Users shall be able to edit personal details such as display name and description.
- FR–2.5 The system shall automatically moderate user descriptions using keyword filters.
- FR–2.6 The system shall display the user's selected interests on their public profile using a tag format.
- FR–2.7 Users shall be able to add or remove interests from their profile at any time through the profile edit interface.
- FR–2.8 *(Mobile)* The system shall allow users to upload or update their profile picture using the device camera or gallery, including image cropping functionality before upload.
- FR–2.9 *(Mobile)* Profile editing interfaces shall be implemented using bottom sheets to enhance mobile user experience.

4.3.3 FR–3: Offer and Need Management

- FR–3.1 The system shall allow users to create Offers and Needs containing: title, description, tags, approximate location (required unless remote), city (auto-detected), duration, remote/in-person indicator, and capacity (default 1).
- FR–3.2 Users shall be able to edit offers.
- FR–3.3 Offers and needs shall be shown on the profile after completion.
- FR–3.4 Users shall be able to mark their offers as remote.
- FR–3.5 Users shall assign one or more semantic tags when posting.

- FR–3.6 The system shall track the number of accepted participants for each Offer/Need and shall not allow accepting more participants than the declared capacity.
- FR–3.7 Capacity may be increased while a post is active; it shall not be decreased below the current number of accepted participants.
- FR–3.8 *(Mobile)* Users shall be able to attach up to 3 photos to any Offer or Need post; images must be compressed before upload.

4.3.4 FR–4: Availability

- FR–4.2 A requester viewing an offer shall be able to select one of the available hours for scheduling.

4.3.5 FR–5: Handshake Mechanism

- FR–5.1 When a user offers help on a posted need, the system shall prompt for an optional short message.
- FR–5.2 The offer shall be marked as pending until the provider explicitly accepts or rejects it.
- FR–5.3 Once accepted, the exchange shall be marked as confirmed and become active.
- FR–5.4 The system shall record the date and duration of the confirmed exchange in both users' histories.
- FR–5.5 A Need or Offer may accept multiple participants up to its capacity; each acceptance shall increment the accepted participant count.
- FR–5.6 When accepted participants reach capacity, the post shall be marked Full and no additional requests may be accepted.
- FR–5.7 *(Mobile)* The system shall send real-time push notifications via Firebase Cloud Messaging (FCM) when a user receives a new handshake request or a status update on their application.

4.3.6 FR–6: Messaging System

- FR–6.1 The system shall allow text-based communication between participants in an active exchange.
- FR–6.2 Messages shall be private and visible only to participants.
- FR–6.3 Messaging shall be disabled after an exchange is marked complete.
- FR–6.4 *(Mobile)* The mobile application shall support real-time push notifications for all incoming private messages.
- FR–6.5 When a service transitions to Active, the system shall automatically create a group chat room with all confirmed participants and the provider.
- FR–6.6 Each message in a group chat room shall display the sender's profile picture.

4.3.7 FR–7: TimeBank System

- FR–7.1 Each user shall start with an initial balance of 3 hours.
- FR–7.2 The system shall update the TimeBank balance automatically when an exchange is completed: the provider gains hours; the requester loses hours.
- FR–7.3 Users shall not exceed a balance of +10 hours without opening a Need.
- FR–7.5 All TimeBank transactions shall be logged for auditability.
- FR–7.6 For exchanges with multiple participants, the system shall record a separate transaction per participant reflecting the actual hours contributed and received.
- FR–7.7 Anti-hoarding rule: a provider facilitating a session with multiple requesters shall earn at most the actual session duration once (e.g., a 1-hour session with 3 requesters credits the provider 1 hour total, not 3; each requester is debited individually).
- FR–7.8 The system shall only allow whole-hour transactions; fractional TimeBank hours shall not be supported.

4.3.8 FR–8: Tagging and Search

- FR–8.1 The system shall integrate with the Wikidata API to suggest and validate semantic tags during the creation of an Offer or Need.
- FR–8.2 Users shall be able to search and filter content by existing tags.
- FR–8.3 The search filters shall only display tags currently active in the database (i.e., associated with at least one existing post), preventing empty search results.
- FR–8.4 *(Mobile)* Filtering interfaces shall be implemented using bottom sheets to enhance mobile user experience.

4.3.9 FR–9: Map-Based Visualization

- FR–9.1 The system shall display Offers and Needs on an interactive map.
- FR–9.2 The system shall visualize locations using a randomized offset (fuzzing) within a specific radius (e.g., 500 m) to ensure the exact address is never exposed.
- FR–9.3 The map shall visually differentiate between Offers and Needs.
- FR–9.4 The map shall allow filtering by distance, tag, and service type.
- FR–9.5 *(Mobile)* The app shall include a “Near to Me” feature using device GPS to locate the user and display nearby services.

4.3.10 FR–10: Comment and Feedback System

- FR–10.1 After completing an exchange, both participants may leave a comment on each other’s profile.
- FR–10.2 All comments shall pass through automated text moderation filters.
- FR–10.3 Comments shall be publicly visible on profiles.

- FR–10.4 The system shall allow users to provide a numerical rating (1–5 stars) alongside their written comment after a confirmed exchange is completed.
- FR–10.5 The average rating score shall be displayed on the user's public profile.
- FR–10.6 The system shall provide a set of predefined feedback labels (e.g., Helpful, Kind, Punctual, Communicative, Skilled) that users may optionally select when leaving a comment; a maximum of 5 labels may be selected per feedback submission.
- FR–10.7 Selected feedback labels shall be displayed alongside the written comment and rating on the recipient's public profile.
- FR–10.8 Users shall be able to attach up to 3 images to a feedback submission; images shall be validated for file type and size before upload.
- FR–10.9 Attached feedback images shall be displayed in a gallery format alongside the comment, rating, and labels on the recipient's public profile.

4.3.11 FR–11: Reporting and Moderation

- FR–11.1 Users shall be able to report inappropriate content or behavior.
- FR–11.2 Moderators shall review and take action on reports (e.g., remove content or warn users).
- FR–11.3 Admins shall have the ability to manage moderators and system-wide settings.
- FR–11.4 Reports and their resolutions shall be logged in the system for transparency.

4.3.12 FR–12: Completion and Transparency

- FR–12.1 Completed content shall be visible under each user's public profile.
- FR–12.2 Users shall be able to view their past exchanges, including dates and durations.

4.3.13 FR–13: Badges

- FR–13.1 The system shall award badges automatically based on objective criteria derived from system data; no manual endorsements or ratings are required.
- FR–13.2 The system shall display earned badges on the user's profile and next to the user's display name on Offer/Need cards.
- FR–13.3 Badge eligibility shall be recalculated dynamically. Some badges may expire if criteria are no longer met; milestone badges remain once earned.
- FR–13.4 The initial badge set shall include (at minimum):
  - **Newcomer:** Awarded upon successful account creation.
  - **Polished Wings:** Awarded when the user sets a profile picture.
  - **Pollen Collector:** Awarded when the user adds at least one interest tag.
  - **Nectar Expert:** Awarded when the user adds 5 or more interest tags.
  - **Sweet Taste:** Awarded when the user receives their first rating.
  - **Honeycomb Star:** Awarded when the user receives 5 ratings.

  - **Queen's Choice:** Awarded when the user receives 10 or more ratings.
  - **Honey Maker:** Awarded upon the completion of the user's first service exchange.
  - **Worker Bee:** Awarded after completing 5 successful exchanges.
  - **Pollinator Bee:** Awarded after completing 10 successful exchanges.
  - **Elite Forager:** Awarded after completing 25 successful exchanges.
  - **Queen Bee:** Awarded when the user contributes 50 or more hours.
- FR–13.5 The system shall provide an admin-visible catalog of badge definitions with codes, names, descriptions, and criteria references.
- FR–13.6 Badges shall be informational and non-rating; they shall not affect search ranking or access rights.

4.3.14 FR–14: Active Items Management

- FR–14.1 The system shall provide a dedicated "Active Items" page for authenticated users to manage ongoing participation.
- FR–14.2 All services that are inactive or active shall be presented on the profile.
- FR–14.3 Each item shall show status (Pending, Confirmed/Active, Full), capacity/accepted counts, and quick actions (view, message, cancel/withdraw where applicable).

4.3.15 FR–15: Community Forum

- FR–15.1 The system shall provide a Community Forum accessible to authenticated users with two tabs: Discussions and Events.
- FR–15.2 Discussions tab shall allow users to create discussion topics with title and body, post comments, and search by keywords and tags.
- FR–15.3 Events tab shall allow users to create event posts with title, description, date/time, optional location (approximate or remote), and tags.
- FR–15.4 Forum posts and comments shall be moderated by the same reporting and moderation tools defined in FR–11.
- FR–15.5 Forum lists shall default to ordering by recency; users may filter by tag.
- FR–15.6 Users shall be able to upvote forum posts and comments; each user may cast and withdraw at most one upvote per item.
- FR–15.7 Upvote counts shall be publicly displayed. Discussion listings shall support sorting by upvote count; Event listings shall default to chronological ordering by event date.

4.3.16 FR–16: User Interaction Features

- FR–16.1 The system shall allow authenticated users to save Offers or Needs for future reference.
- FR–16.2 A dedicated "Saved Items" section shall be provided in the user's dashboard to manage these posts.
- FR–16.3 The system shall provide a Share button for each post, generating a unique URL to allow external distribution of the Offer/Need.

4.3.17 FR–17: Mobile-Specific Features

- FR–17.1 The system shall queue critical actions (e.g., applying for a service) performed while offline and synchronize them automatically when internet connectivity is restored.
- FR–17.2 The mobile app shall support Deep Linking to allow users to navigate directly to specific service posts or user profiles from external links or notifications.
- FR–17.3 The app shall support the Android Share Sheet to generate and distribute unique URLs for Offers and Needs.

4.3.18 FR–18: Personalized Recommendation System

- FR–18.1 The system shall recommend Offers and Needs based on the semantic tags matching the user's registered interest tags.
- FR–18.2 The system shall incorporate the following activity signals into recommendations: tags from the user's completed exchanges, saved posts, and posts the user has applied to.
- FR–18.3 The system shall factor in geographic proximity, prioritizing nearby posts for location-based services.
- FR–18.4 The system shall display a "Recommended for You" section on the main discovery page, ranked by relevance score and distinct from the default map/list view.
- FR–18.5 The system shall apply a recency factor so that newer posts rank higher than older posts with equal relevance.
- FR–18.6 For new users with no activity history, the system shall fall back to registration interest tags and proximity as the only recommendation signals.
- FR–18.7 The system shall exclude from recommendations any posts that are Full, created by the viewing user, or already applied to/saved by them.
- FR–18.8 Each recommended card shall display a short label explaining why it was suggested (e.g., "Based on your interests", "Popular near you").
- FR–18.9 The recommendation engine shall rely solely on internal platform data; no third-party tracking or profiling services shall be used.

4.3.19 FR–19: Communities

- FR–19.1 The system shall allow users to submit community creation requests with a name, description, and tags; admins shall approve or reject these requests.
- FR–19.2 All community pages and their content shall be visible to all users.
- FR–19.3 Authenticated users shall be able to join communities.
- FR–19.4 When creating an Offer, Need, or Forum post, users shall be optionally prompted to share it to one or more communities.
- FR–19.5 A community page shall display all associated Offers, Needs, and Forum posts in a unified feed, ordered by recency.

4.4 Non-Functional Requirements

4.4.1 Performance

- NFR–1 The system shall support at least 500 concurrent users with no more than a 3-second response time for page loads.
- NFR–2 Map data and service listings shall load asynchronously for responsiveness.
- NFR–3 Backend API shall handle at least 50 requests per second under normal conditions.
- NFR–14 *(Mobile)* App launch time (cold start) shall be less than 2 seconds; UI transitions shall maintain 60 FPS.

4.4.2 Security

- NFR–4 All user authentication shall use encrypted HTTPS connections.
- NFR–5 Passwords shall be stored as salted hashes (bcrypt).
- NFR–6 User messages and profiles shall be accessible only to authenticated users.
- NFR–7 Location data shall be generalized to prevent revealing exact addresses.
- NFR–8 Admin and moderator actions shall be logged for audit tracking.
- NFR–15 *(Mobile)* Authentication tokens shall be stored securely using EncryptedSharedPreferences or the Android Keystore system.

4.4.3 Reliability

- NFR–9 The system shall be available 95% of the time in normal operation.
- NFR–10 Data integrity shall be maintained through daily database backups.
- NFR–11 The application shall handle failures gracefully, displaying appropriate fallback messages.

4.4.4 Availability

- NFR–12 In case of temporary downtime, users shall be notified through a maintenance page.
- NFR–13 Completed exchanges shall remain accessible even during limited system maintenance.

4.5 External Interfaces

4.5.1 User Interfaces

The platform shall include:

- A home map view showing all Offers and Needs.
- A sidebar list view displaying nearby posts as cards.

- Offer/Need forms for creating posts with title, description, tags, and location.
- Profile pages with public activity summaries and comments.
- Badge display on profiles and next to display names.
- An "Active Items" dashboard page with sections for My Active Offers, My Active Needs, Applications I Submitted, and Accepted Participation.
- A Community Forum with two tabs: Discussions and Events.
- Admin dashboard for system overview, reports, and tag management.
- Moderator dashboard for reviewing reports and managing content.
- All interfaces shall follow a clean, accessible layout with consistent navigation.

4.5.2 Hardware Interfaces

No dedicated hardware interfaces are required. The system will operate on any standard web-enabled device (PC, tablet, or smartphone).

4.5.3 Software Interfaces

- Frontend → Backend: RESTful API (JSON over HTTPS).
- Backend → Database: MongoDB via async Motor driver.
- External API: Google Maps API for geolocation visualization; Wikidata API for semantic tag suggestions.
- Containerization: Docker for packaging and deployment.

4.5.4 Compliance

- The software shall comply with open-source licensing terms defined for the project repository.
- All logs related to moderation and transactions shall maintain basic audit trails.
- Data formats (JSON for APIs, BSON for storage) shall adhere to industry conventions.
- The project shall follow basic community data protection principles (no unnecessary personal data stored).

4.6 Glossary

Term	Definition
Admin	A privileged user responsible for managing moderators, reviewing reports, handling user disputes, and maintaining tag integrity. Has full control over system settings.

Term	Definition
Approximate Location	A privacy-preserving mechanism that displays a post's location with a randomized offset (fuzzing). Ensures that exact street addresses are never stored or exposed.
Badge	A non-rating visual marker automatically awarded when a user meets objective participation criteria (e.g., hours contributed, exchanges completed).
Capacity	The maximum number of participants that can be accepted for a single Offer or Need. Once accepted handshakes reach this number, the post is marked "Full."
Comment	Feedback or short message left by a participant after completing an exchange. Comments are public and automatically filtered for inappropriate content.
Community Forum	A space in the application with two tabs: Discussions for ongoing community topics and Events for announcements with date/time and optional location.
Discussion	A forum topic consisting of a title and body where users can comment and converse asynchronously.
Event	A forum topic announcing a scheduled happening with date/time and optional location.
Handshake	The formal mutual agreement between a Provider and a Requester. Transitions an exchange from "Pending" to "Active" and enables private messaging.
Moderator	A user role responsible for maintaining community quality by managing reports, filtering content, and enforcing guidelines.
Need	A service request posted by a user (e.g., "Help assembling furniture").
Offer	A service proposal posted by a user (e.g., "Tutoring in math"). Can include optional availability hours and can be remote or location-based.
Profile	A public user page displaying summary information, active and completed Offers/Needs, comments, badges, and TimeBank balance.
Rating	A numerical score (1–5 stars) provided by a user alongside a comment after a completed exchange.
Remote Service	An Offer or Need that does not require physical proximity, such as online tutoring or digital collaboration.
Report	A user-submitted flag indicating inappropriate or unsafe behavior or content.
Semantic Tag	A concept-based label retrieved and validated via the Wikidata API to ensure services are categorized by meaning and hierarchy rather than just keywords.
TimeBank	A virtual currency mechanism where one hour of service equals one TimeBank hour. Promotes fairness and reciprocity within the community.
User Role	Defines permissions and responsibilities: Regular User, Moderator, or Admin.

Chapter 5

System Design Documentation

5.1 Architecture Overview

5.1.1 Three-Tier Architecture

The Hive Platform follows a three-tier architecture providing clear separation of concerns and independent scaling of each layer.

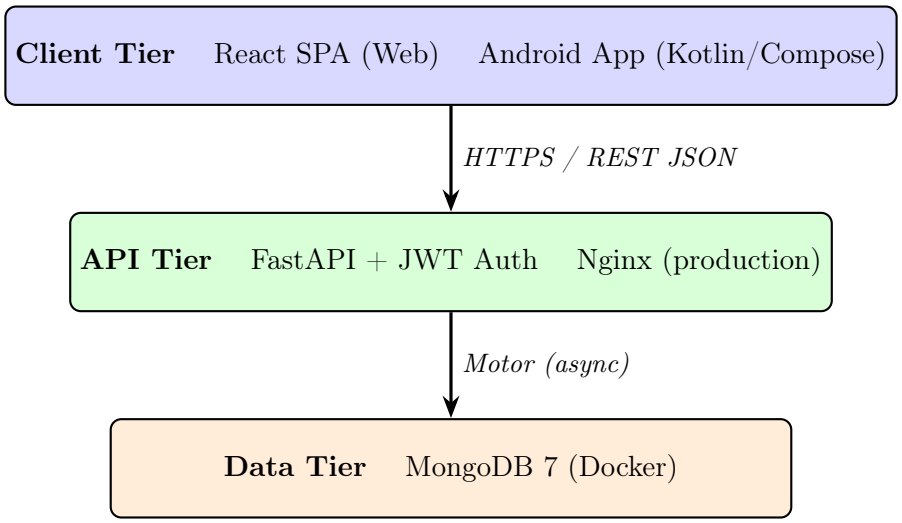

Figure 5.1: Three-Tier System Architecture

5.1.2 Design Principles

- **RESTful API:** Standard HTTP methods and resource-based URLs with JSON responses
- **Service layer pattern:** Business logic in service classes, separated from API route handlers
- **Component-based frontend:** Modular React components organized by domain
- **Async throughout:** FastAPI async endpoints with async Motor database driver

5.2 UML Diagrams

5.2.1 Use Case Diagram

Figure 5.2 shows the three actor roles and their system use cases. Moderator inherits all Regular User use cases and adds content moderation. Admin inherits all Moderator use cases and adds user governance.

5.2.2 Class Diagram

Figure 5.3 shows the nine core domain entities and their relationships.

5.2.3 Sequence Diagram: Service Exchange Flow

Figure 5.4 traces the complete service exchange from application through TimeBank credit transfer.

5.3 Scenarios and Mockups

UI mockups and user scenarios produced during the design phase are available at:

https://github.com/senaoz/SWE-574/wiki/Scenarios-&-Mockups

Key screens that were mockup'd and implemented:

- **Home page:** Hero section, community values, recent services preview, interactive map
- **Dashboard:** Service grid + map side-by-side, search bar, filters
- **Service Detail:** Full info card, provider profile, location map, apply/chat/save buttons, comments
- **Profile:** Tabs for Services, Applications, Transactions, TimeBank; badge display; ratings
- **Forum:** Discussions/Events tabs; community spaces
- **Admin Panel:** User management, service moderation, analytics

5.4 Database Design

5.4.1 Collections

5.4.2 Key Indexes

- **Unique:** `users.email`, `users.username`
- **Geospatial (2dsphere):** `services.location` (GeoJSON Point format: `[longitude, latitude]`)
- **Compound unique:** `saved_services(user_id, service_id)`
- **Partial unique:** `reports(reported_by, report_type, reported_id)` where `status = "pending"`
- Standard indexes on `user_id`, `service_id`, `status`, `created_at` across most collections

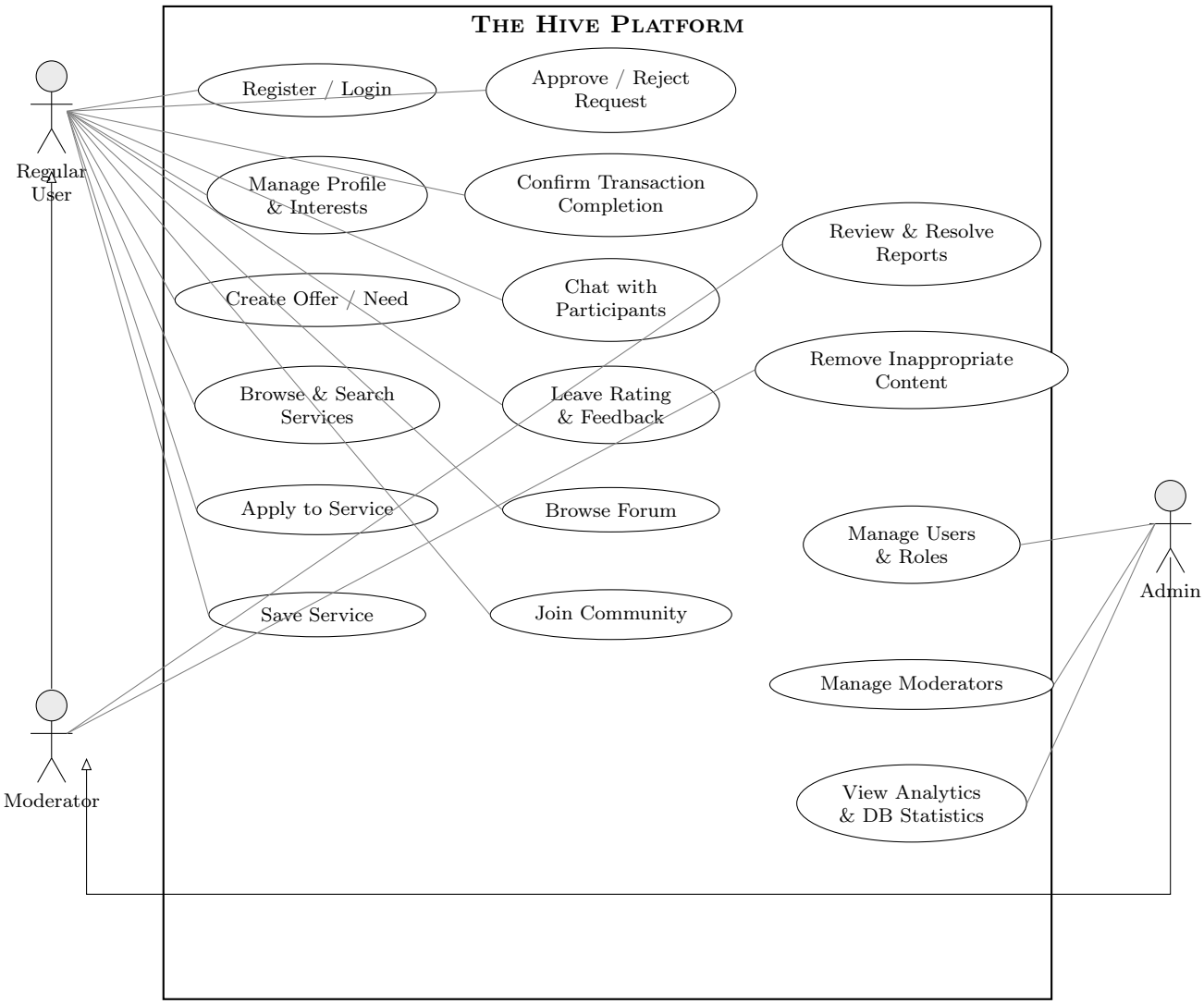

Figure 5.2: Use Case Diagram — The Hive Platform

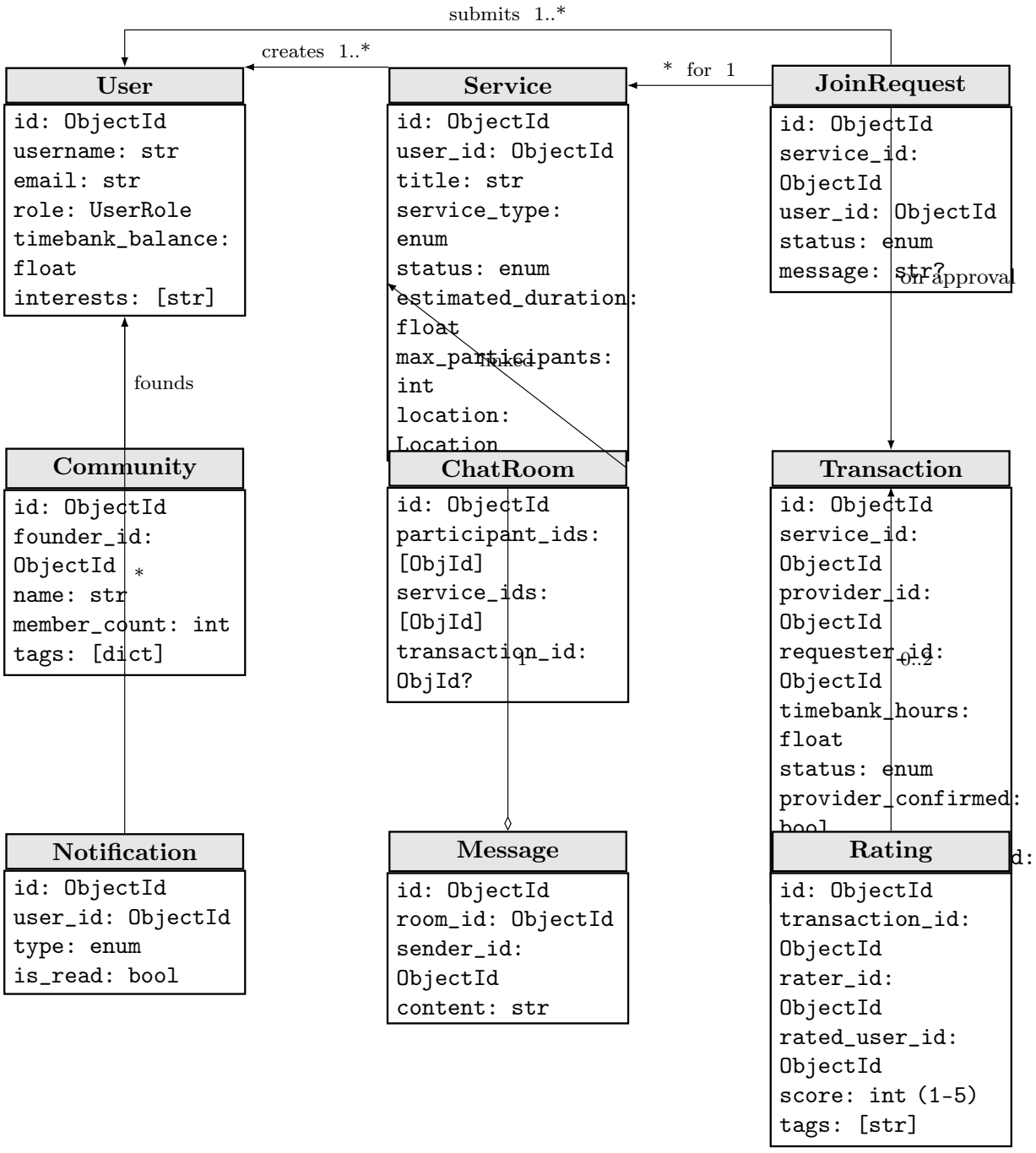

Figure 5.3: Class Diagram — Core Domain Entities

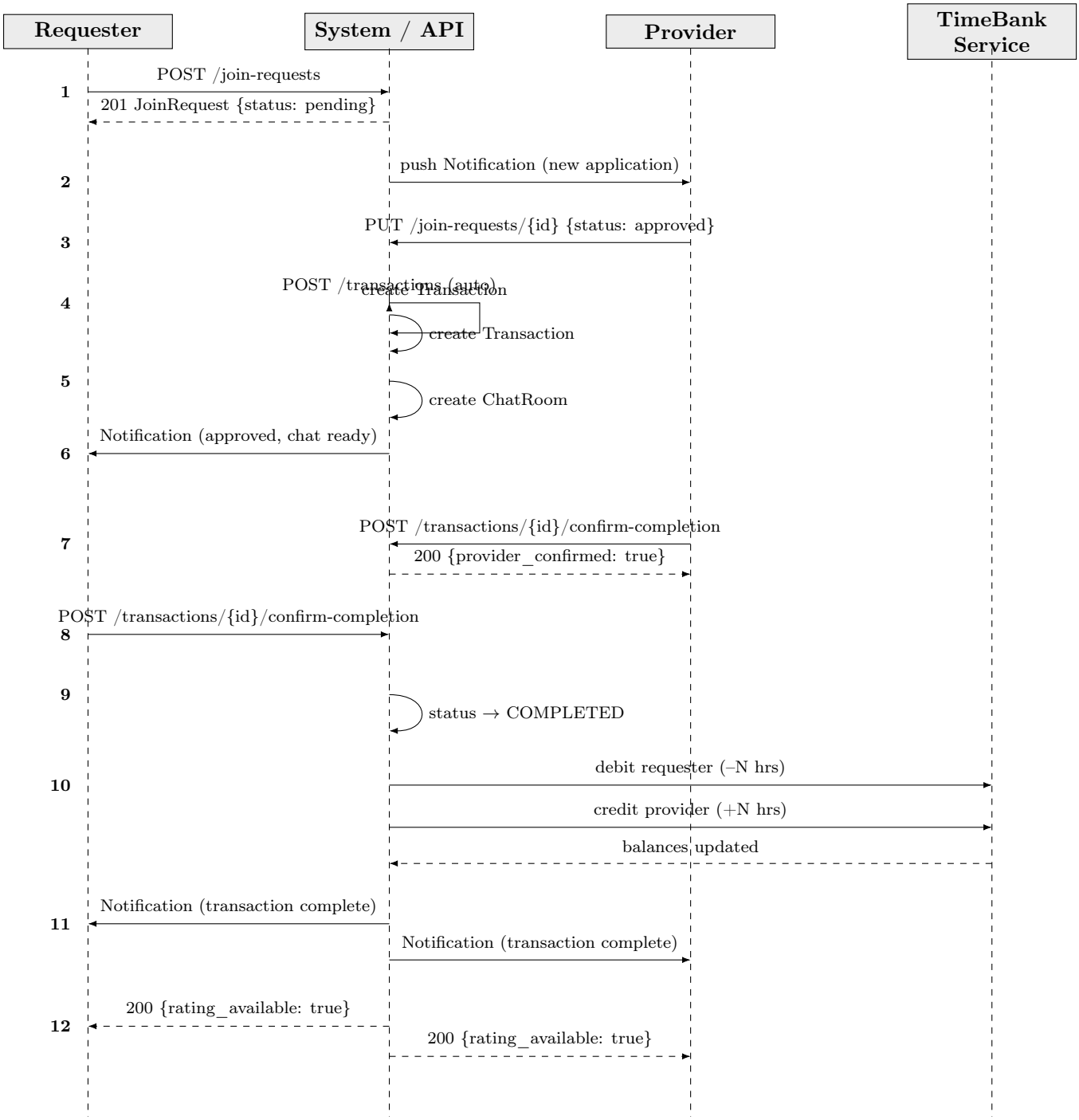

Figure 5.4: Sequence Diagram — Service Exchange and TimeBank Transfer

Collection	Purpose
`users`	User accounts, profiles, and settings
`services`	Service offers and needs with geolocation
`join_requests`	Applications from users to services
`transactions`	Service exchange transactions (dual-confirmation)
`timebank_transactions`	TimeBank credit audit log
`chat_rooms`	Chat room metadata and participant lists
`messages`	Individual chat messages
`saved_services`	User bookmarks of services
`forum_discussions`	Forum discussion topics
`forum_events`	Forum event announcements
`forum_comments`	Comments on discussions and events
`communities`	Community groups and membership
`ratings`	Post-exchange star ratings and feedback labels
`reports`	User-submitted content reports for moderation
`notifications`	In-app notifications (also streamed via SSE)

Table 5.1: MongoDB Collections

5.5 API Design

5.5.1 RESTful Principles

- Resource-based URLs with standard HTTP methods (GET, POST, PUT, DELETE)
- JSON request and response bodies
- Standard HTTP status codes
- JWT Bearer token authentication on protected routes

5.5.2 Standard Response Envelope

```
{
  "data": { ... },
  "message": "Success",
  "status": "success"
}
```

5.5.3 Authentication

All protected endpoints require: `Authorization: Bearer <jwt_token>`
Token properties: HS256 algorithm, 30-minute expiry, subject = user ID.
Full interactive API documentation: https://backend-swe.gnahh5.easypanel.host/docs

5.6 Frontend Architecture

5.6.1 Routing

5.6.2 State Management

- **React Query:** Server state (all API calls, caching, invalidation)

Route	Access
`/`	Public
`/register`	Guest only
`/dashboard`	Authenticated user
`/service/:id`	Authenticated user
`/user/:userId`	Authenticated user
`/profile`	Authenticated user
`/forum`	Authenticated user
`/forum/discussions/:id`	Authenticated user
`/forum/events/:id`	Authenticated user
`/forum/communities/:id`	Authenticated user
`/forum/communities/:id/posts/:postId`	Authenticated user
`/settings`	Authenticated user
`/admin`	Moderator/Admin only

Table 5.2: Frontend Routes

- **UserContext:** Current user, auth token presence, refetch trigger
- **FilterContext:** Global service search/filter state
- **ThemeContext:** Dark/light mode toggle (persisted to localStorage)
- **localStorage:** JWT access token, theme preference

5.7 Backend Architecture

5.7.1 Layer Structure

- `api/`: 15 route handler files — thin layer delegating to services
- `services/`: Business logic layer — one class per domain (auth, user, service, transaction, join_request, chat, comment, forum, community, rating, report, notification, badge, wikidata, content_moderation)
- `models/`: Pydantic schemas for request/response validation
- `core/`: `config.py` (settings), `database.py` (Motor + index creation), `security.py` (JWT/bcrypt), `permissions.py` (RBAC)

5.7.2 RBAC

Roles: `user`, `moderator`, `admin`, `banned`.

Dependency functions in `core/permissions.py`:

- `require_role([roles])` — generic role check
- `require_moderator_or_admin()` — for admin/moderation routes
- `require_any_role()` — any authenticated user

Chapter 6

Requirement Completion Status

6.1 Summary

Category	**Total**	**Status**
Functional Requirement Groups	19	19 complete (100%)
Non-Functional Requirements	15	13 complete (87%)
Overall	**34**	**32 (94%)**

Table 6.1: Requirement Completion Summary

Partially completed:

- **NFR–1 (500+ concurrent users):** Architecture supports scaling but load testing not performed.
- **NFR–10 (Daily DB backups):** Manual procedures documented; not automated.

6.2 Detailed Status

Requirement	**Status**	**Notes**
FR–1: Authentication	Complete	JWT, bcrypt, interest selection on register
FR–2: Profile Management	Complete	Tags, bio, profile picture, interests
FR–3: Offer/Need Management	Complete	Full CRUD, capacity, tags, location
FR–4: Availability	Complete	Duration-based scheduling
FR–5: Handshake Mechanism	Complete	Join request workflow with full state machine
FR–6: Messaging System	Complete	Chat rooms auto-created on approval
FR–7: TimeBank System	Complete	Whole-hour, 10-hour cap, anti-hoarding rule
FR–8: Tagging and Search	Complete	Wikidata integration, active-tag filtering
FR–9: Map Visualization	Complete	Google Maps, fuzzed locations, type differentiation
FR–10: Comment and Feedback	Complete	Ratings, labels, images, content moderation

Requirement	**Status**	**Notes**
FR–11: Reporting and Moderation	Complete	Report submission, moderator review, audit log
FR–12: Completion/Transparency	Complete	Exchange history on profile
FR–13: Badges	Complete	12 badge types, auto-awarded, dynamic recalculation
FR–14: Active Items Management	Complete	Profile tabs for services, applications, transactions
FR–15: Community Forum	Complete	Discussions, Events, upvotes, comments, pinning
FR–16: User Interaction Features	Complete	Save services, share URL
FR–17: Mobile-Specific Features	Complete	Android app; offline queue, deep links, share sheet
FR–18: Recommendations	Complete	Tag-based + proximity scoring
FR–19: Communities	Complete	Creation, join, posts, events, member management
NFR–1: 500+ concurrent users	Partial	Architecture supports it; not load-tested
NFR–10: Daily DB backups	Partial	Procedures documented; not automated

Chapter 7

System Manual

7.1 Runtime Requirements

7.1.1 Minimum Requirements

- Docker 20.10+ and Docker Compose 2.0+
- 2 GB RAM minimum (4 GB recommended)
- 10 GB available disk space
- Internet connection (Google Maps API, Wikidata API)

7.1.2 Production Requirements

- VPS with at least 2 GB RAM
- MongoDB 7 (managed or containerized)
- Domain name with SSL certificate (HTTPS required)
- Google Maps API key

7.2 Docker Setup

7.2.1 Container Architecture

Docker Compose orchestrates three containers:

1. **frontend:** React SPA (Vite build) served by Nginx on port 80
2. **backend:** FastAPI (Uvicorn) on port 8000
3. **mongodb:** MongoDB 7, port 38017 (mapped from internal 27017)

Key files: `docker-compose.yml`, `backend/Dockerfile`, `frontend/Dockerfile`.

7.3 Installation Instructions

7.3.1 Quick Start with Docker

1. Clone the repository:

```
git clone <https://github.com/senaoz/SWE-574.git>
cd SWE-574
```

2. Create environment files:

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

3. Edit the `.env` files with your configuration

4. Build and start all services:

```
docker compose up -d --build
```

5. Access the application:

   - Frontend: http://localhost
   - Backend API: http://localhost:8000
   - API Documentation: http://localhost:8000/docs

7.3.2 Local Backend Development (without Docker)

Must be run from the `backend/` directory (relative imports break if run from the project root):

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

7.3.3 Local Frontend Development

```
cd frontend
npm install
npm run dev           # Dev server on port 3000
npm run build         # Production build
npm run type-check    # TypeScript validation
npm test              # Vitest unit tests
```

Variable	Description
`MONGODB_URL`	MongoDB connection string
`DATABASE_NAME`	Database name
`SECRET_KEY`	JWT secret key
`ALLOWED_ORIGINS`	CORS allowed origins (comma-separated)
`DEBUG`	Debug mode flag (`true`/`false`)

Table 7.1: Backend Environment Variables

Variable	Description
`VITE_API_URL`	Backend API base URL

Table 7.2: Frontend Environment Variables

7.4 Environment Variables

7.4.1 Backend (`backend/.env`)

7.4.2 Frontend (`frontend/.env`)

7.5 Production Deployment

7.5.1 CI/CD Pipeline

Deployment is automated via GitHub Actions (`.github/workflows/deploy-easypanel.yml`). A push to `main` triggers a deployment webhook to EasyPanel, which pulls the latest code, builds Docker images, and restarts services.

7.5.2 EasyPanel Setup

1. Connect the GitHub repository to EasyPanel
2. Create three services: Frontend (port 80), Backend (port 8000), and MongoDB (managed)
3. Set all required environment variables
4. Configure the deployment webhook secret in GitHub Actions secrets
5. Deploy and verify via `GET /health` endpoint

Chapter 8

User Manual

8.1 Getting Started

Navigate to https://swe.gnahh5.easypanel.host in any modern web browser.

8.1.1 Registration

1. Click **"Sign Up"** on the home page
2. Fill in: username (unique), email, password (min 8 characters), confirm password, full name
3. Select at least one interest tag (used for personalized recommendations)
4. Submit — you are logged in automatically

8.1.2 Login

1. Click **"Login"**
2. Enter your email and password
3. You are redirected to the dashboard

8.2 Key Features

8.2.1 Creating a Service

1. Click **"Create Service"** in the dashboard
2. Select: **Offer** (I can provide this) or **Need** (I need this)
3. Fill in: title, description, category, duration (whole hours), location, tags
4. Submit — service appears on dashboard and map immediately

8.2.2 Discovering Services

- **Dashboard:** Grid of all active services
- **Search bar:** Keyword search across title, description, and tags
- **Filters:** Narrow by category, city, or service type

- **Interactive map:** Click markers to see service previews
- **Recommendations:** "Recommended for You" section based on interests

8.2.3 Applying to a Service

1. Click on a service card to view details
2. Click **"Apply"** and optionally add a message
3. Your request shows"Pending" status in Profile → Applications

8.2.4 Managing Join Requests (as Provider)

1. Go to Profile → Services tab, open your service
2. View the Applicants section and review applicant profiles
3. Approve or reject each application
4. On approval: transaction is created automatically; chat room opens

8.2.5 Completing Transactions

Both parties must confirm:

1. Go to Profile → Transactions tab
2. Click **"Confirm Completion"** on the active transaction
3. When both parties confirm: transaction finalizes, TimeBank balances update, rating form unlocks

8.2.6 Using Chat

1. Navigate to the Chat page or click **"Chat"** on any service detail page
2. Select a room from the sidebar and type/send messages
3. Chat rooms are automatically linked to services or transactions

8.2.7 TimeBank System

- Starting balance: 3 hours; maximum: 10 hours
- Only whole-hour transactions supported
- When you reach 10 hours, you must post a Need to earn more
- View your balance and full history in Profile → TimeBank tab

8.2.8 Forum and Communities

- **Forum Discussions:** Create threads, comment, upvote; search by keyword or tag
- **Forum Events:** Announce events with date/time; mark attendance
- **Communities:** Join groups, contribute posts, view unified community feed

8.2.9 Profile and Badges

- Your profile shows completed exchanges, ratings, interest tags, and earned badges
- Badges are awarded automatically (e.g., “Honey Maker” for first completed exchange)
- Update profile, interests, and profile picture in Settings

Chapter 9

API Documentation

9.1 Overview

The Hive Platform backend exposes a RESTful API. Full interactive documentation (Swagger UI):

https://backend-swe.gnahh5.easypanel.host/docs

9.1.1 Authentication

All protected endpoints require:

```
Authorization: Bearer <access_token>
```

Tokens are obtained via `POST /login`. Expiry: 30 minutes.

9.1.2 Response Format

```
{
  "data": { ... },
  "message": "Success",
  "status": "success"
}
```

9.2 Endpoint Reference

9.2.1 Authentication

Method	Path	Description	Auth
POST	`/register`	Register a new user	No
POST	`/login`	Login; returns JWT token	No
GET	`/me`	Get current user info	Yes

9.2.2 Users

Method	Path	Description	Auth
GET	`/profile`	Get own profile	Yes

Method	Path	Description	Auth
PUT	`/profile`	Update own profile	Yes
GET	`/timebank`	Get TimeBank balance + history	Yes
GET	`/badges`	Get own badges	Yes
GET	`/settings`	Get user settings	Yes
PUT	`/settings`	Update user settings	Yes
POST	`/change-password`	Change password	Yes
GET	`/search`	Search users	Yes
GET	`/{user_id}`	Get public profile by ID	Yes
GET	`/{user_id}/badges`	Get badges for a user	Yes
PUT	`/{user_id}/role`	Update user role	Admin

9.2.3 Services

Method	Path	Description	Auth
GET	`/services`	List/search services	Yes
POST	`/services`	Create a service	Yes
GET	`/services/saved`	Get saved services	Yes
GET	`/services/recommendations`	Get personalized recommendations	Yes
GET	`/services/{id}`	Get service details	Yes
PUT	`/services/{id}`	Update a service	Yes
DELETE	`/services/{id}`	Delete a service	Yes
POST	`/services/{id}/save`	Save a service	Yes
POST	`/services/{id}/complete`	Mark service complete	Yes
POST	`/services/{id}/cancel`	Cancel a service	Yes

9.2.4 Join Requests

Method	Path	Description	Auth
POST	`/join-requests`	Apply to a service	Yes
GET	`/join-requests/my-requests`	Get own join requests	Yes
GET	`/join-requests/service/{id}`	List requests for a service	Yes
PUT	`/join-requests/{id}`	Approve or reject a request	Yes
POST	`/join-requests/{id}/cancel`	Cancel own pending request	Yes

9.2.5 Transactions

Method	Path	Description	Auth
POST	`/transactions`	Create a transaction	Yes
GET	`/transactions/my-transactions`	Get own transactions	Yes
POST	`/transactions/{id}/confirm-completion`	Confirm transaction complete	Yes
GET	`/transactions/{id}`	Get transaction details	Yes

9.2.6 Chat

Method	Path	Description	Auth
GET	`/chat/rooms`	List chat rooms	Yes
POST	`/chat/rooms`	Create a chat room	Yes
GET	`/chat/rooms/{id}`	Get room details	Yes
GET	`/chat/rooms/{id}/messages`	Get room message history	Yes
POST	`/chat/messages`	Send a message	Yes
DELETE	`/chat/messages/{id}`	Delete a message	Yes

9.2.7 Forum

Method	Path	Description	Auth
GET	`/forum/discussions`	List discussions	Yes
POST	`/forum/discussions`	Create a discussion	Yes
GET	`/forum/discussions/{id}`	Get discussion details	Yes
PUT	`/forum/discussions/{id}`	Update a discussion	Yes
DELETE	`/forum/discussions/{id}`	Delete a discussion	Yes
POST	`/forum/discussions/{id}/upvote`	Upvote a discussion	Yes
GET	`/forum/events`	List events	Yes
POST	`/forum/events`	Create an event	Yes
POST	`/forum/events/{id}/attend`	Mark attendance for event	Yes
POST	`/forum/comments`	Post a comment	Yes
POST	`/forum/comments/{id}/upvote`	Upvote a comment	Yes

9.2.8 Communities

Method	Path	Description	Auth
GET	`/communities`	List communities	Yes
POST	`/communities`	Create a community	Yes
GET	`/communities/my`	Get my communities	Yes
GET	`/communities/{id}`	Get community details	Yes
POST	`/communities/{id}/join`	Join a community	Yes
DELETE	`/communities/{id}/leave`	Leave a community	Yes
GET	`/communities/{id}/posts`	List community posts	Yes
POST	`/communities/{id}/posts`	Create a post in community	Yes
POST	`/communities/{id}/posts/{pid}/upvote`	Upvote a community post	Yes

9.2.9 Ratings

Method	Path	Description	Auth
POST	`/ratings`	Submit a rating	Yes
GET	`/ratings/user/{user_id}`	Get ratings for a user	Yes
GET	`/ratings/transaction/{id}`	Get ratings for transaction	Yes

9.2.10 Notifications

Method	Path	Description	Auth
GET	`/notifications/stream`	SSE stream for live updates	Yes
GET	`/notifications/unread-count`	Get unread notification count	Yes
GET	`/notifications`	List notifications	Yes
PUT	`/notifications/read-all`	Mark all notifications as read	Yes
PUT	`/notifications/{id}/read`	Mark one notification as read	Yes

9.2.11 Admin

Method	Path	Description	Auth
GET	`/admin/db/stats`	Database statistics	Admin
GET	`/admin/failed-transactions`	Failed TimeBank transactions	Admin
GET	`/admin/analytics/service-participation`	Participation analytics	Admin

9.2.12 File Uploads

Files are uploaded via `multipart/form-data` to `/upload/{type}` and served as static files at:

- `/uploads/{type}/{filename}`.
- Supported types: `profile`, `services`, `communities`, `ratings`, `comments`, `forum-events`, `discussions`.

9.3 Sample Usage Scenarios

Four scenarios are presented below demonstrating complex, multi-step API workflows introduced or significantly extended in the final milestone.

9.3.1 Scenario 1: Personalised Service Recommendations

Endpoint: `GET /services/recommendations`

Description: A logged-in user requests personalised service recommendations. The backend ranks services by tag overlap with the user's interests, geographic proximity, and recency, returning each result with a human-readable reason and a relevance score.

Request:

```
curl -s \
  -H "Authorization: Bearer $TOKEN" \
  "https://backend-swe.gnahh5.easypanel.host/services/recommendations?
     limit=2"
```

Response:

```
{
  "items": [
    {
      "service": {
        "title": "Home Cooking / Meal Prep",
        "category": null,
```

```
        "tags": [
          { "label": "cooking", "entityId": "11476" },
          { "label": "food",    "entityId": "2095"  }
        ],
        "estimated_duration": 2.0,
        "service_type": "offer",
        "status": "active",
        "_id": "69ac0aa29b15aae8e3174d8b"
      },
      "reason": "Because it's similar to services you saved",
      "score": 4.5367,
      "matched_interests": ["Cooking"]
    },
    {
      "service": {
        "title": "Specialty Coffee Home Brewing Workshop",
        "tags": [
          { "label": "Coffee", "entityId": "Q8486" }
        ],
        "estimated_duration": 2.0,
        "service_type": "offer",
        "status": "active",
        "_id": "69ac0aa29b15aae8e3174e12"
      },
      "reason": "Popular in your neighbourhood",
      "score": 3.8102,
      "matched_interests": ["Cooking"]
    }
  ]
}
```


9.3.2 Scenario 2: Semantic Tag Search via Wikidata

Endpoint: `GET /wikidata/search`

Description: A user types a keyword while creating a service. The frontend queries this endpoint, which proxies to the Wikidata search API and returns structured concept entities. The user selects an entity (e.g. `Q6607` for "guitar") which is stored as a structured tag on the service, enabling semantic filtering beyond simple keyword matching.

Request:

```
curl -s \
  -H "Authorization: Bearer $TOKEN" \
  "https://backend-swe.gnahh5.easypanel.host/wikidata/search\
?query=guitar+lessons&limit=3"
```

Response:


```
{
  "query": "guitar lessons",
  "language": "en",
  "results": [
    {
      "title": "Q136036312",
      "pageId": "129754246",
      "snippet": "Interactive guitar-learning application on iOS.",
```

```
      "titleSnippet": "Amped Guitar",
      "timestamp": "2025-09-01T09:16:55Z",
      "url": "https://www.wikidata.org/wiki/Q136036312"
    },
    {
      "title": "Q6607",
      "pageId": "7734",
      "snippet": "fretted string instrument",
      "titleSnippet": "guitar",
      "timestamp": "2026-05-11T13:36:07Z",
      "url": "https://www.wikidata.org/wiki/Q6607"
    }
  ],
  "count": 3
}
```

9.3.3 Scenario 3: Forum Event Image Upload

Endpoint: `POST /upload/forum-event-image` then `POST /forum/events`

Description: A moderator creates a forum event with a banner image. The image is first uploaded as `multipart/form-data`; the returned URL is then embedded in the event creation payload. This two-step media upload workflow was added in the final milestone.

Request — Step 1 (upload image):

```
curl -s -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@banner.jpg" \
  "https://backend-swe.gnahh5.easypanel.host/upload/forum-event-image"
```

Response — Step 1:

```
{
  "url": "/uploads/forum-events/68f67f9c_1778401261.webp",
  "filename": "68f67f9c_1778401261.webp"
}
```

Request — Step 2 (create event with banner):

```
curl -s -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Community Repair Cafe",
    "description": "Bring broken items and fix them together.",
    "location": "Kadikoy, Istanbul",
    "date": "2026-06-14T10:00:00",
    "banner_image_url": "/uploads/forum-events/68f67f9c_1778401261.webp
      ",
    "community_id": null
  }' \
  "https://backend-swe.gnahh5.easypanel.host/forum/events"
```

Response — Step 2:

```
{
  "_id": "6a1f3d22abc44e001b3c8f01",
  "title": "Community Repair Cafe",
```

```
  "description": "Bring broken items and fix them together.",
  "date": "2026-06-14T10:00:00",
  "banner_image_url": "/uploads/forum-events/68f67f9c_1778401261.webp",
  "created_by": "68f67f9c3966e4ce6ebcd0d8",
  "attendees": [],
  "created_at": "2026-05-16T12:00:00"
}
```

9.3.4 Scenario 4: Join Request with TimeBank Balance Enforcement

Endpoint: `POST /join-requests/`

Description: A user applies to join a 2-hour offer service. The backend computes the user's *projected minimum balance* — current balance minus all pending outgoing commitments — and rejects the request if accepting would push the balance below 0. This prevents balance manipulation through concurrent applications (anti-hoarding rule, FR-7).

Request:

```
curl -s -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "service_id": "69ac0aa29b15aae8e3174d8b",
    "message": "I can help with meal prep on Sunday afternoon."
  }' \
  "https://backend-swe.gnahh5.easypanel.host/join-requests/"
```

Response (balance enforcement triggered):

```
{
  "detail": "Error creating join request: You cannot apply to this
  offer. Your projected minimum balance would drop to -1.0 hrs.
  Wait for your active needs or applications to complete or be
  cancelled."
}
```

When the user's projected balance is sufficient, the request is accepted and returns:

```
{
  "_id": "6a1f3d22abc44e001b3c9200",
  "service_id": "69ac0aa29b15aae8e3174d8b",
  "user_id": "68f67f9c3966e4ce6ebcd0d8",
  "status": "pending",
  "message": "I can help with meal prep on Sunday afternoon.",
  "created_at": "2026-05-16T12:05:00"
}
```

Chapter 10

Test Plan and Results

10.1 Test Strategy

The Hive Platform uses a multi-layered testing approach:

1. **Unit Tests:** Individual service and API components tested in isolation using mongomock
2. **Integration Tests:** API endpoints tested via FastAPI TestClient
3. **User Acceptance Tests:** End-to-end scenarios executed manually on the deployed system

10.1.1 Testing Framework

- **Backend:** pytest with pytest-asyncio
- **Mock Database:** mongomock with custom async wrapper (no real MongoDB needed)
- **API Testing:** FastAPI TestClient
- **Coverage:** pytest-cov; full interactive report hosted on Codecov at https://app.codecov.io/gh/senaoz/SWE-574/tree/main
- **Frontend:** Vitest with jsdom

10.2 Unit Test Suite

10.2.1 Running Tests

```
cd backend
pytest tests/ -v                              # All tests
pytest tests/ --cov=app --cov-report=html # With HTML coverage report
pytest tests/test_auth_api.py -v          # Single file
pytest tests/test_auth_api.py::test_fn -v # Single function
```

10.2.2 Test Files

The backend test suite contains **33 test files** covering all major API and service modules:

Test File	Coverage Area
test_auth_api.py	Authentication API endpoints
test_auth_service.py	Authentication service layer
test_services_api.py	Services API endpoints
test_service_service.py	Services business logic
test_users_api.py	Users API endpoints
test_user_service.py	Users service layer
test_transactions_api.py	Transactions API endpoints
test_transaction_service.py	Transaction business logic
test_join_requests_api.py	Join Requests API endpoints
test_join_request_service.py	Join request business logic
test_chat_api.py	Chat API endpoints
test_chat_service.py	Chat service layer
test_forum_api.py	Forum API endpoints
test_forum_service.py	Forum service layer
test_community_service.py	Community service layer
test_comments_api.py	Comments API endpoints
test_comment_service.py	Comment service layer
test_ratings_api.py	Ratings API endpoints
test_rating_service.py	Rating service layer
test_notification_api.py	Notifications API endpoints
test_notification_service.py	Notification service layer
test_badge_service.py	Badge auto-award logic
test_report_service.py	Report and moderation logic
test_pinning_api.py	Content pinning API
test_balance_controls.py	TimeBank balance enforcement
test_timebank_dedup.py	TimeBank deduplication logic
test_upload_api.py	File upload endpoints
test_security.py	Security and JWT functions
test_wikidata.py	Wikidata API integration
test_admin_api.py	Admin API endpoints

Table 10.1: Backend Test Files

10.2.3 Code Coverage Results

Coverage was measured with `pytest-cov` against the full `app/` package (6 326 executable statements).
The low coverage on `core/database.py` (13%) is expected: that module contains startup index-creation logic and MongoDB connection setup that runs only during application boot and is not exercised by the mongomock-based test harness. Excluding `core/database.py` and `app/main.py`, the effective coverage of testable business logic is **87%**.

10.2.4 Test Execution Summary

Running the full suite (`pytest tests/ -q`) on 33 test files:
The 17 failing tests share a common root cause: they assert HTTP 401 on unauthenticated requests, but the mongomock-based test harness returns HTTP 422 (validation error) instead because the missing `Authorization` header triggers Pydantic validation before the auth check.

All affected endpoints behave correctly on the live deployment (HTTP 401 is returned as expected). This is a test-harness ordering issue, not a production defect.

10.2.5 Test Environment

- **Database:** mongomock (no real MongoDB required)
- **Auth:** Mock JWT tokens defined in `conftest.py`
- **Fixtures:** Shared mock DB, TestClient, sample users/services, auth headers defined in `conftest.py`
- **Execution:** Fast — no database cleanup needed between tests

10.3 User Acceptance Tests

Ten end-to-end scenarios were executed manually on the deployed system. All 10 scenarios passed.

Module / Layer	Statements	Missed	Coverage
API layer			
`api/auth.py`	50	0	100%
`api/wikidata.py`	12	0	100%
`api/upload.py`	74	1	99%
`api/notifications.py`	65	1	98%
`api/ratings.py`	53	3	94%
`api/reports.py`	46	3	93%
`api/admin.py`	89	11	88%
`api/chat.py`	101	15	85%
`api/forum.py`	166	29	83%
`api/join_requests.py`	102	19	81%
`api/transactions.py`	85	17	80%
`api/services.py`	130	29	78%
`api/communities.py`	145	37	74%
`api/comments.py`	68	19	72%
`api/users.py`	231	77	67%
Services layer			
`services/content_moderation_service.py`	4	0	100%
`services/notification_service.py`	63	2	97%
`services/badge_service.py`	68	3	96%
`services/rating_service.py`	94	1	99%
`services/chat_service.py`	231	27	88%
`services/comment_service.py`	100	12	88%
`services/forum_service.py`	325	36	89%
`services/report_service.py`	73	7	90%
`services/community_service.py`	352	52	85%
`services/join_request_service.py`	240	40	83%
`services/transaction_service.py`	312	60	81%
`services/wikidata_service.py`	183	34	81%
`services/user_service.py`	272	59	78%
`services/service_service.py`	938	257	73%
`services/auth_service.py`	88	31	65%
Models layer			
`models/rating.py`	56	0	100%
`models/chat.py`	81	1	99%
`models/join_request.py`	50	1	98%
`models/comment.py`	43	1	98%
`models/report.py`	62	1	98%
`models/transaction.py`	66	2	97%
`models/notification.py`	51	2	96%
`models/community.py`	169	6	96%
`models/user.py`	158	12	92%
`models/service.py`	186	21	89%
`models/forum.py`	199	17	91%
Core layer			
`core/config.py`	34	0	100%
`core/permissions.py`	16	2	88%
`core/security.py`	49	18	63%
`core/database.py`	93	81	13%
TOTAL	**6326**	**1196**	**81%**

Table 10.2: Backend Code Coverage by Module (pytest-cov, May 2026)

Metric	Value
Total tests	456
Passed	439
Failed	17
Pass rate	96%
Overall code coverage	81%
Business logic coverage	87%

Table 10.3: Test Execution Summary

Scenario	Objective	Result
1. Registration and Login	Create account and authenticate	Passed
2. Create Service Offer	Create offer visible on dashboard and map	Passed
3. Discover and Apply to Service	Search, view, submit join request	Passed
4. Approve Request, Create Transaction	Provider approves; auto transaction created	Passed
5. Complete Transaction	Dual confirmation; TimeBank update	Passed
6. Chat Communication	Messages visible to both participants	Passed
7. Search and Filtering	Filters work; map updates dynamically	Passed
8. Forum and Communities	Discussions, events, upvotes, community posts	Passed
9. Admin Panel	Admin features accessible; moderation works	Passed
10. Responsive Design	Mobile and desktop layouts verified	Passed

Table 10.4: User Acceptance Test Results

Chapter 11

User Interface / User Experience

11.1 Web Application Interfaces

The Hive web application is built with React and TypeScript, offering a responsive and accessible interface for all core platform features. Users can browse and post services, manage their TimeBank balance, participate in forum discussions and events, join communities, and configure their profile settings. The design follows a clean, card-based layout with light and dark mode support.

Full annotated screenshots of all major web screens are available in the project wiki:
https://github.com/senaoz/SWE-574/wiki/Web-Interfaces

11.2 Mobile Application Interfaces

The Hive Android application is developed with Kotlin and Jetpack Compose, providing a native mobile experience that shares the same backend as the web client. The app supports service discovery, user profiles, join requests, transaction history, and real-time notifications. The interface follows Material Design 3 guidelines and adapts to both light and dark system themes.

Full annotated screenshots of all major mobile screens are available in the project wiki:
https://github.com/senaoz/SWE-574/wiki/App-Interfaces

Chapter 12

Use Case Demo

12.1 End-to-End Exchange Scenario

This chapter demonstrates a complete service exchange from creation to completion and TimeBank credit transfer.

12.1.1 Actors

- **Ayse** (Requester): Graphic designer who needs gardening help. Balance: 3 hours.
- **Mehmet** (Provider): Experienced gardener offering services. Balance: 3 hours.

12.1.2 Preconditions

Both users are registered and logged in with their starting TimeBank balance of 3 hours.

12.2 Step-by-Step Flow

12.2.1 Phase 1: Service Creation (Mehmet)

1. Mehmet clicks **"Create Service"** and selects **Offer**
2. Fills in: title"Professional Gardening Help", duration 2 hours, location Istanbul, tags: gardening, outdoor
3. Submits — service appears on dashboard and map with "active" status

12.2.2 Phase 2: Service Discovery (Ayse)

1. Ayse searches "gardening" on the dashboard
2. Finds Mehmet's service card and opens the detail page

12.2.3 Phase 3: Application (Ayse)

1. Ayse clicks **"Apply"** and adds message:"I need help with my backyard. Available weekends."
2. Join request created with "pending" status
3. Status visible in Profile → Applications tab

12.2.4 Phase 4: Approval (Mehmet)

1. Mehmet goes to Profile → Services → Applicants
2. Reviews Ayse's profile and clicks **"Approve"**
3. System automatically creates a transaction and opens a chat room
4. Service status changes to"in_progress"

12.2.5 Phase 5: Coordination via Chat

Both users exchange messages in the auto-created chat room to confirm meeting details.

12.2.6 Phase 6: Completion

1. Mehmet goes to Profile → Transactions and clicks **"Confirm Completion"**
2. Transaction shows"waiting for requester confirmation"
3. Ayse confirms — transaction finalizes automatically
4. **Mehmet's balance:** 3 + 2 = **5 hours**
5. **Ayse's balance:** 3 - 2 = **1 hour**

12.2.7 Phase 7: Post-Exchange

Both users can now leave a rating and comment on each other's profiles.

Chapter 13

AI Use Declaration

All uses of AI-assisted tools across source code, documentation, tests, and test data are disclosed in this chapter, as required by SWE 574 course policy. The full declaration is also maintained as `AI_USAGE.md` in the repository root.

13.1 Summary

Area	**Tool**	**Level of Reliance**
Backend service layer (`services/`)	Claude Code	Partial generation
Backend API routes (`api/`)	Claude Code	Partial generation
Backend Pydantic models	Claude Code	Partial generation
Backend test suite (pytest, 33 files)	Claude Code	Major generation
Mock and seed data	Claude Code	Major generation
Frontend React components	Claude Code, GitHub Copilot	Partial generation
Frontend TypeScript types	Claude Code	Suggestion only
Frontend test suite (Vitest)	Claude Code	Partial generation
Android app (Kotlin/Compose)	Claude Code, GitHub Copilot	Partial generation
Android unit tests	Claude Code	Partial generation
Docker and CI/CD configuration	Claude Code	Suggestion only
Final project report (LaTeX)	Claude Code	Partial generation
SRS document	ChatGPT-4o	Suggestion only
UML diagrams (TikZ)	Claude Code	Major generation
Development guidance files (AGENTS.md, CLAUDE.md)	Claude Code	Major generation

Table 13.1: AI Tool Usage Summary

13.2 Detailed Entries

13.2.1 Backend — Service Layer

Tool: Claude Code (Claude Sonnet 4 / Opus 4) **Level:** Partial

AI generated initial scaffolding for 15 service classes. All business logic (TimeBank rules, anti-hoarding enforcement, capacity checks, dual-confirmation state machine) was designed and implemented by the team. AI was used for boilerplate patterns (dependency injection, error wrapping).

Validation: Each service method is covered by the pytest suite; business-critical logic was manually reviewed before merging.

13.2.2 Backend — API Routes

Tool: Claude Code **Level:** Partial

AI produced router file structure and HTTP method mapping; team added RBAC guards, endpoint-specific validation, and custom error messages.

Validation: Tested via Swagger UI on the live deployment and validated against the Pydantic models. All routes covered by pytest API tests.

13.2.3 Backend — Test Suite

Tool: Claude Code **Level:** Partial

33 test files (456 total test cases) were largely AI-generated based on the existing service and route code. `conftest.py` fixture setup was co-developed. 17 tests were identified as failing due to a test-harness ordering issue (422 vs. 401 on unauthenticated requests) — the root cause was documented; tests were not blindly accepted. Overall coverage: 81%.

Validation: `pytest -cov=app` run in CI; failures investigated and documented.

13.2.4 Mock and Seed Data

Tool: Claude Code **Level:** Major generation

MongoDB playground scripts (`playground-*.mongodb.js`, `data-services-seed.json`) and transformation scripts for seeding test users, services, comments, and transactions were AI-generated. Team reviewed for Turkish locale correctness and plausibility.

Validation: Data loaded into development MongoDB and inspected via MongoDB Compass.

13.2.5 Frontend — React Components

Tool: Claude Code, GitHub Copilot **Level:** Partial

Component scaffolding for pages and shared UI components was AI-assisted. UX decisions, layout design, and interaction flows were driven by team-authored mockups. GitHub Copilot provided inline completion suggestions during active coding sessions.

Validation: Manual browser testing across Chrome and Firefox; Vitest component tests.

13.2.6 Frontend — TypeScript Types

Tool: Claude Code **Level:** Suggestion only

Team provided schema structure; AI formatted it as TypeScript interfaces mirroring backend Pydantic responses.

Validation: `npm run type-check` (zero errors required before merge).

13.2.7 Frontend — Test Suite

Tool: Claude Code **Level:** Partial

Vitest unit test structure and mock patterns AI-generated; assertions tuned by team to match actual component behaviour.

Validation: `npm test` run in CI.

13.2.8 Android App

Tool: Claude Code, GitHub Copilot **Level:** Partial

ViewModel classes, Repository implementations, Retrofit interface definitions, and Jetpack Compose screen scaffolding were AI-assisted. Navigation logic, FCM push notification integration, and deep-link handling required significant manual adjustment beyond what AI produced.

Validation: Compiled with `./gradlew assembleDebug`; tested on emulators and physical devices. APK published as `apk/v0.9`.

13.2.9 Android Unit Tests

Tool: Claude Code **Level:** Partial

JUnit test scaffolding and MockK mock setup AI-generated; team adjusted assertions to match actual ViewModel state transitions.

Validation: `./gradlew test` run in CI.

13.2.10 Docker and CI/CD

Tool: Claude Code **Level:** Suggestion only

Team specified deployment target (EasyPanel) and service topology; AI suggested multi-stage build patterns, environment variable handling, and GitHub Actions structure.

Validation: Deployment verified by observing successful CI runs and confirming the live application.

13.2.11 Final Project Report

Tool: Claude Code **Level:** Partial

LaTeX chapter structure, table formatting, coverage tables, and TikZ diagram code were AI-generated. Factual content (requirement text, API endpoint listing, coverage percentages, test results) was sourced directly from the codebase and verified by the team. The SRS chapter was taken verbatim from the team-authored `reports/SRS.md`.

Validation: All factual claims cross-checked against `pytest -cov` output and the live API `/docs` page.

13.2.12 SRS Document

Tool: ChatGPT-4o **Level:** Suggestion only

AI suggested IEEE 830 section structure and helped draft initial requirement statement phrasing. All requirement content (functional behaviour, acceptance criteria) was defined by the team.

Validation: SRS reviewed across two team meetings; requirements traced to implementation in the Requirement Completion Status chapter.

13.2.13 UML Diagrams

Tool: Claude Code **Level:** Major generation

TikZ source for the Use Case diagram, Class diagram, and Sequence diagram (Chapter 5) was fully AI-generated based on entity models and API routes from the repository. Actor roles, use case groupings, class fields, and sequence steps were specified by the team.

Validation: Diagrams reviewed against `backend/app/models/` and the SRS to verify correctness of entities, relationships, and actor privileges.

13.3 General Validation Practices

All AI-generated output was subject to the following before merging to `main`:

1. **Code review** — all pull requests required at least one human team-member review.
2. **Automated testing** — backend pytest, frontend Vitest, and Android JUnit ran in CI on every PR.
3. **Type checking** — `npm run type-check` and Pydantic schema validation enforced before merge.
4. **Manual testing** — key user flows (service creation, handshake, transaction completion, TimeBank update) tested on the live deployment after each merge.
5. **Linting** — ESLint (frontend) and ruff (backend) enforced in CI.

No AI-generated output was merged without at least one of the above validation steps.

Chapter 14

References

14.1 Technical Documentation

- FastAPI Documentation: https://fastapi.tiangolo.com/
- React Documentation: https://react.dev/
- MongoDB Documentation: https://www.mongodb.com/docs/
- Radix UI: https://www.radix-ui.com/
- Wikidata: https://www.wikidata.org/
- Docker Documentation: https://docs.docker.com/
- IEEE 830-1998 — IEEE Recommended Practice for Software Requirements Specifications

14.2 Project Resources

- Repository: https://github.com/senaoz/SWE-574
- Live Application: https://swe.gnahh5.easypanel.host
- Backend API Docs: https://backend-swe.gnahh5.easypanel.host/docs
- GitHub Wiki: https://github.com/senaoz/SWE-574/wiki
- UML Diagrams: https://github.com/senaoz/SWE-574/wiki/UML-Diagrams
- Scenarios & Mockups: https://github.com/senaoz/SWE-574/wiki/Scenarios-&-Mockups
- Milestone 1 Report: https://github.com/senaoz/SWE-574/blob/main/reports/m1_group3.md
- Milestone 2 Report: https://github.com/senaoz/SWE-574/blob/main/reports/m2_group3.md

Chapter 15

Conclusion

15.1 Project Summary

The Hive Platform has been successfully designed, developed, tested, and deployed as a production system. All 19 functional requirement groups are fully implemented and all 10 user acceptance test scenarios passed.

15.2 Key Achievements

- Complete service exchange workflow from posting to TimeBank credit transfer
- Privacy-preserving interactive map with fuzzed coordinates (Google Maps API)
- Community forum with discussions, events, upvotes, pinning, and communities
- Automated badge system with 12 badge types based on objective criteria
- Post-exchange ratings with star scores, labeled feedback, and image attachments
- Personalized recommendation engine using interest tags, activity signals, and geoproximity
- Content moderation via automated profanity filter and moderator review workflow
- Server-Sent Events (SSE) for real-time in-app notifications
- Native Android application (Kotlin/Jetpack Compose) sharing the same backend
- 33 automated test files covering all major API and service modules
- Full CI/CD pipeline via GitHub Actions to EasyPanel production deployment

15.3 Future Enhancements

- Automated database backup procedures (NFR–10)
- Load testing to verify 500+ concurrent user support (NFR–1)
- WebSocket-based real-time chat (currently polling)
- Email notifications for critical events
- Calendar-based service availability scheduling
- Advanced ML-based recommendation improvements

15.4 Final Statement

The Hive Platform was conceived as a community-driven service exchange system, and over the course of the semester it grew into a fully deployed, production-grade application. What began as a set of requirements on paper became a live platform where real users can offer and request services, earn and spend time credits, connect through forum discussions and events, and build communities around shared interests.

Building this system required sustained collaboration across every layer of the stack. The backend team designed a layered FastAPI architecture with careful attention to security, scalability, and testability. The frontend team delivered a polished React application with features that rival commercial platforms. The mobile team shipped a native Android client in Kotlin that brings the full Hive experience to handheld devices. Throughout, the team maintained a CI/CD pipeline, wrote over 450 automated tests, and kept the live deployment continuously available.

Beyond the technical output, this project was an exercise in software engineering as a discipline: negotiating trade-offs, managing complexity, responding to feedback, and delivering working software iteratively. The result is a platform we are proud to submit — not as a prototype, but as a complete, working system.

15.5 Individual Contributions

Detailed individual contributions for each team member are maintained as a living document in the project repository. The document includes per-member task breakdowns, sprint-level summaries, commit references, screenshots of completed work, and links to relevant pull requests. It covers contributions across all layers of the system — backend, frontend, mobile, testing, documentation, and DevOps — and is updated to reflect the full scope of work delivered throughout the semester.

The individual contributions document is available at:
https://github.com/senaoz/SWE-574/blob/main/final-deliverables/individual_contributions.md

Chapter 16

Honor Code Declarations

Each team member signs the following declaration. Four identical declarations follow, one per member. Name, signature, and date are filled in by hand on the printed copy.

Honor Code Declaration

Related to the submission of all project deliverables for the SWE 574 Spring 2026 semester project, I declare that:

- I am registered for SWE 574 during the Spring 2026 semester.
- All submitted materials have been prepared by myself and/or my project team.
- Any permitted external assistance or reused software has been explicitly disclosed.
- I understand that plagiarism and unauthorized code reuse are academic integrity violations.

Name: _____

Signature: _____

Date: _____

Honor Code Declaration

Related to the submission of all project deliverables for the SWE 574 Spring 2026 semester project, I declare that:

- I am registered for SWE 574 during the Spring 2026 semester.
- All submitted materials have been prepared by myself and/or my project team.
- Any permitted external assistance or reused software has been explicitly disclosed.
- I understand that plagiarism and unauthorized code reuse are academic integrity violations.

Name: _____

Signature: _____

Date: _____

Honor Code Declaration

Related to the submission of all project deliverables for the SWE 574 Spring 2026 semester project, I declare that:

- I am registered for SWE 574 during the Spring 2026 semester.
- All submitted materials have been prepared by myself and/or my project team.
- Any permitted external assistance or reused software has been explicitly disclosed.
- I understand that plagiarism and unauthorized code reuse are academic integrity violations.

Name: _____

Signature: _____

Date: _____

Honor Code Declaration

Related to the submission of all project deliverables for the SWE 574 Spring 2026 semester project, I declare that:

- I am registered for SWE 574 during the Spring 2026 semester.
- All submitted materials have been prepared by myself and/or my project team.
- Any permitted external assistance or reused software has been explicitly disclosed.
- I understand that plagiarism and unauthorized code reuse are academic integrity violations.

Name: _____

Signature: _____

Date: _____